5

Creating a Photo Editor Using CameraX

In the smartphone era, taking and sharing photos has become second nature, and platforms such as Instagram have shown us how powerful a single photo can be. For apps like these, it's not just about snapping a picture; it's about enhancing and personalizing that image to tell a story. But have you ever wondered what lies behind those in-app camera buttons and filters?

Enter CameraX, Android's go-to tool for everything camera-related. This tool doesn't just make capturing photos seamless; it's also the bridge to editing and refining them. In this chapter, we'll get hands-on with CameraX, discovering how it can transform the Packtagram photography experience. We'll also design an interactive space for users to tweak and enhance their shots, adding that personal touch. And for the cherry on top? We'll dive into a bit of smart tech, teaching our app to recognize photo themes and suggest relevant hashtags.

Building on our prior work – crafting the screens and feed for our Instagram-inspired app – we're now diving deeper into the app's features. With CameraX, intuitive editing tools, and some clever features, we're set to elevate our app's photo-sharing game.

In this chapter, we will cover the following topics:

- Getting to know CameraX
- Integrating CameraX into our Packtagram app
- Adding photo-editing functionalities
- Using **machine learning** (**ML**) to categorize photos and generate hashtags

Technical requirements

As in the previous chapter, you will need to have Android Studio (or another editor of your preference) installed.

You can find the complete code that we will be using in this chapter in this book's GitHub repository: **https://github.com/PacktPublishing/Thriving-in-Android-Development-using-Kotlin/tree/main/Chapter-5**.

Getting to know CameraX

Since the inception of the Android platform, cameras have played a pivotal role in defining the feature set of smartphones. From capturing moments to enabling augmented reality experiences, the camera has evolved from a mere hardware component to a powerful tool for developers. This evolution, however, has not been without its complexities.

The evolution of camera libraries in Android

Since the first version of Android, developers interacted with the camera hardware through the Camera API; this was Android's first attempt at giving developers the power to harness the capabilities of onboard cameras.

As devices proliferated and features such as more advanced photo hardware grew, the need for a more robust API became evident. Consequently, Camera2 API was introduced in API level 21 (Lollipop). While this offered more granular control over camera capabilities and supported the expanding features of new hardware, its steep learning curve made camera development challenging for many in terms of complexity and performance overhead.

Given the intricacies of Camera2 and the variances in camera hardware across different devices, developers found it increasingly difficult to provide a consistent camera experience to end users. This fragmentation, alongside the complexity of Camera2, made it imperative for a more streamlined, developer-friendly solution.

Enter CameraX.

The importance and advantages of CameraX

CameraX is Android's modern solution for camera app development that was developed with the primary goal of simplifying the process while reducing the fragmentation between devices. Here's why it has quickly become indispensable:

- **Consistency across devices**: CameraX abstracts the differences between device-specific camera behaviors, ensuring that most features work consistently across a wide range of devices.
- **Life cycle awareness**: Gone are the days of tedious life cycle management. CameraX is integrated with Android's life cycle libraries, meaning less boilerplate code and more focus on core camera functionality.
- **Use case-based approach**: Instead of dealing with low-level tasks, developers can now focus on specific use cases, such as image preview, image capture, and image analysis. This makes development faster and less error-prone.
- **Extensions for enhanced capabilities**: With the CameraX Extensions API, developers can access device-specific features such as portrait

mode, HDR, and more, further enriching the camera experience.

- **Backward compatibility**: CameraX offers compatibility with devices running Android 5.0 (API level 21) and beyond, ensuring a wider reach than Camera2.
- **Performance and quality**: CameraX provides optimized performance out of the box, delivering high-quality images and videos without the need for extra tuning.

In summary, CameraX has not only simplified camera app development but also bridged the gap that's caused by hardware discrepancies. As we delve deeper into this chapter, you'll come to appreciate the nuances and capabilities that CameraX brings to the table, setting the stage for powerful, consistent, and high-quality camera applications on Android.

Now, let's start using CameraX and configuring its dependencies in our project.

Setting up CameraX

To set up CameraX, we need to add the necessary dependencies to our version catalog file, **`libs.versions.toml`**, as follows:

```
[versions]
...
camerax = "1.2.1"
accompanist = "0.31.1-alpha"
[libraries]
...
cameraCore = { module = "androidx.camera:camera-core", version.ref = "camerax" }
cameraCamera2 = { module = "androidx.camera:camera-camera2", version.ref = "camerax" }
cameraView = { module = "androidx.camera:camera-view", version.ref = "camerax" }
cameraExtensions = { module = "androidx.camera:camera-extensions", version.ref = "camerax" }
accompanist = { group = "com.google.accompanist", name = "accompanist-permissions", version.ref
```

In this code block, we are adding the dependencies that are needed to use CameraX, plus a library called Accompanist.

Accompanist is a collection of extension libraries that are designed to complement Jetpack Compose. It fills the gaps by offering utilities for specific use cases and easing the integration of Compose with other Android capabilities. The features of Accompanist include image loading integrations, useful components such as ViewPager, tools to manage system UI insets, Compose navigation enhancements, and permissions handling. To learn more and expand on this information, please refer to the official documentation: **https://google.github.io/accompanist/**.

In our case, we are going to use it to simplify the process of checking and asking the user for camera permissions.

Regarding the dependencies to use CameraX, we are adding the following:

- **`cameraCore`**: This dependency provides the core functionality of CameraX, including the ability to manage camera devices, configure capture sessions, and receive frames from the camera. It is the foundation for all other CameraX dependencies.
- **`cameraCamera2`**: This dependency provides the Camera2 implementation of CameraX, which is the most powerful and flexible way to access the camera on Android devices. It provides low-level access to the camera's hardware and allows for custom capture configurations and processing pipelines.
- **`cameraView`**: This dependency provides a pre-built view component that integrates with CameraX to simplify the process of displaying camera preview frames. It takes care of the layout and setup of the view so that you can focus on capturing and processing the camera data.
- **`cameraExtensions`**: This dependency provides a set of extensions for CameraX that add additional features, such as support for focus peaking, image stabilization, and panorama capture. It also includes extensions for working with ML models on camera frames.

NOTE

The versions in the previous code are the latest stable ones at the time of writing this book, but there will likely be new ones by the time you are reading this.

After adding these dependencies to the version catalog, we need to add them to the **`build.gradle.kts`** file of the **`:feature:stories`** module, as follows:

```
    implementation(libs.cameraCore)
    implementation(libs.cameraCamera2)
    implementation(libs.cameraView)
    implementation(libs.cameraExtensions)
    implementation(libs.androidx.camera.lifecycle)
    implementation(libs.accompanist)
```

Now that our project is ready to use CameraX, let's learn more about the library.

Learning about CameraX's core concepts

In this section, we'll learn about some of CameraX's most important concepts.

View life cycle

CameraX, a Jetpack support library, simplifies camera development across Android devices, and its life-cycle-aware nature seamlessly integrates with Jetpack Compose, empowering developers to create resilient and efficient camera applications. At the core of CameraX's design philosophy lies its inherent support for Android's life cycle, which eliminates the complexities of managing camera resources. CameraX automatically

handles camera start, stop, and resource release based on life cycle events, streamlining the development process.

Jetpack Compose, the declarative UI toolkit for Android, is also deeply rooted in life cycle concepts. Composables inherently possess life cycle states, such as **`onActive`** and **`onDispose`**, that get triggered during their existence within the UI hierarchy. Combining the powers of CameraX and Compose offers a harmonized approach to managing the camera's life cycle within Composable UI components.

Image analysis

CameraX goes beyond just capturing images. With **image analysis**, developers can process live camera feeds in real time. This is perfect for features such as barcode scanning, face detection, or even applying live filters. Here is an example:

```
@Composable
fun CameraPreviewWithImageAnalysis() {
    val cameraProvider = rememberCameraProvider()
    val preview = remember { Preview.Builder().build() }
    val text = remember { mutableStateOf("Analyzing...") }
    val imageAnalyzer = ImageAnalysis.Builder()
        .setAnalyzer { image ->
            // Process the image data here
            text.value = "Detected image to analyze..."
        }
        .build()
    LaunchedEffect(cameraProvider) {
        val useCaseBinding = UseCaseBinding.Builder()
            .addUseCases(preview, imageAnalyzer)
            .build()
        val camera =
            cameraProvider.bindToLifecycle(useCaseBinding)
        camera.close()
    }
    Box(modifier = Modifier.fillMaxSize()) {
        Preview(preview)
        Text(text.value)
    }
}
```

The preceding code defines a composable function called **`CameraPreviewWithImageAnalysis`** that displays a camera preview and analyzes the live camera feed, utilizing Jetpack Compose and CameraX to achieve this.

First, the **`rememberCameraProvider`** function is used to retrieve the camera provider instance, which is responsible for managing the camera's life cycle and providing access to camera controls. Then, a **`Preview`** instance is created using **`Preview.Builder`** to define the camera preview surface. This preview will display the live camera feed on the screen.

After that, an **`ImageAnalysis`** instance is created using **`ImageAnalysis.Builder`** to process the live camera feed. The **`setAnalyzer`** method is used to specify an analyzer function that will be called whenever a new image frame is available.

A **`LaunchedEffect`** block is used to start a coroutine that binds the camera preview and image analyzer to the camera's life cycle. The **`bindToLifecycle`** method is used to connect the use cases to the camera's life cycle, ensuring that they start and stop automatically when the app starts and stops.

A **`mutableStateOf`** variable text is used to store the current state of the analysis. The text variable is updated within the analyzer function to reflect the results of the image analysis.

Finally, the **`Box`** composable is used to lay out the camera preview and the text. The **`fillMaxSize`** modifier is used to make **`Box`** occupy the entire screen. The **`Preview`** composable is placed inside **`Box`** to display the camera preview. The **`Text`** composable is also placed inside **`Box`** to display the current state of the analysis.

This is a basic example of how to apply image analysis, but there are some already existing image analyzers, such as **`BarcodeScanner`**. The following code is built upon the previous one, adding this analyzer:

```
@Composable
fun BarcodeScannerPreview() {
    val cameraProvider = rememberCameraProvider()
    val preview = remember { Preview.Builder().build() }
    val barcodeText = remember { mutableStateOf("") }
    val barcodeScanner = BarcodeScanner.Builder()
        .setBarcodeFormats(BarcodeScannerOptions.
            BarcodeFormat.ALL_FORMATS)
        .build()
    LaunchedEffect(cameraProvider) {
        val imageAnalyzer = ImageAnalysis.Builder()
            .setAnalyzer { image ->
                val rotation =
                    image.imageInfo.rotationDegrees
                val imageProxy =
                    InputImage.fromMediaImage(image.image,
                        rotation)
                barcodeScanner.processImage(imageProxy)
                    .addOnSuccessListener { barcodes ->
                        if (barcodes.isNotEmpty()) {
                            val barcode = barcodes[0]
                            barcodeText.value =
                                barcode.displayValue
                        } else {
                            barcodeText.value = "No barcode
                                detected"
                        }
                    }
                    .addOnFailureListener { e ->
```

```
                        barcodeText.value = "Barcode
                            scanning failed: ${e.message}"
                    }
                }
                .build()
            val useCaseBinding = UseCaseBinding.Builder()
                .addUseCases(preview, imageAnalyzer)
                .build()
            val camera =
                cameraProvider.bindToLifecycle(useCaseBinding)
            camera.close()
        }
        Box(modifier = Modifier.fillMaxSize()) {
            Preview(preview)
            Text(barcodeText.value)
        }
    }
```

Similar to the previous example, this code defines a composable function called **`BarcodeScannerPreview`** that displays a camera preview and analyzes the live camera feed for barcodes. However, this code specifically focuses on barcode scanning and utilizes the ML Kit **`BarcodeScanner`** library to achieve this functionality.

First, the **`rememberCameraProvider`** and **`Preview`** functions are used in the same way as they were in the previous example to retrieve the camera provider instance and create a preview instance for displaying the live camera feed.

Then, a **`BarcodeScanner`** instance is created using **`BarcodeScanner.Builder`**, specifying the barcode formats to be detected. In this case, all barcode formats are specified using **`BarcodeScannerOptions.BarcodeFormat.ALL_FORMATS`**.

Following this, an **`ImageAnalysis`** instance is created using **`ImageAnalysis.Builder`**, and the analyzer function is defined to process each image frame. First, the analyzer function retrieves the image rotation from the **`imageInfo`** object. Then, it converts the **`ImageProxy`** instance into an **`InputImage`** format that's compatible with ML Kit's **`BarcodeScanner`**.

The **`BarcodeScanner.processImage`** method is called on the **`InputImage`** instance to detect barcodes. Here, **`OnSuccessListener`** is used to handle the successful barcode detection, while **`OnFailureListener`** is used to handle any errors that occur during barcode scanning.

If barcodes are detected, the **`displayValue`** value of the first barcode is extracted and stored in the **`barcodeText`** mutable state variable. This variable is used to update the text field with the detected barcode information.

With this, we have created our first image analyzer to get barcode information. Let's move on to the next feature: **`CameraSelector`**.

CameraSelector

When dealing with cameras, it's not always about just one camera – many modern devices come with multiple camera lenses. This is where **`CameraSelector`** comes to the rescue, allowing developers to programmatically choose between, say, the front or rear camera. Whether you're building a selfie app or a more standard photo application, **`CameraSelector`** ensures consistent behavior across the board. Let's see how we can allow a user to select which camera they want to use:

```
@Composable
fun CameraSelectorExample() {
    val cameraProvider = rememberCameraProvider()
    val preview = remember { Preview.Builder().build() }
    val isUsingFrontCamera = remember {
        mutableStateOf(true) }
    val cameraSelector = remember {
        if (isUsingFrontCamera.value) {
            CameraSelector.DEFAULT_FRONT_CAMERA
        } else {
            CameraSelector.DEFAULT_BACK_CAMERA
        }
    }
    val imageAnalyzer = ImageAnalysis.Builder()
        .setAnalyzer { image ->
            // Process the image data here
        }
        .build()
    LaunchedEffect(cameraProvider) {
        val useCaseBinding = UseCaseBinding.Builder()
            .addUseCases(preview, imageAnalyzer)
            .build()
        val camera =
            cameraProvider.bindToLifecycle(useCaseBinding)
        camera.close()
    }
    Box(modifier = Modifier.fillMaxSize()) {
        Preview(preview)
        Column {
            Button(onClick = {
                isUsingFrontCamera.value =
                    !isUsingFrontCamera.value
            }) {
                Text("Switch Camera")
            }
        }
    }
}
```

The preceding code will display a camera preview and a button. Clicking the button will switch between the front and rear cameras. The **`isUsingFrontCamera`** mutable state variable is used to keep track of which camera is currently being used. Then, **`cameraSelector`** is updated whenever the **`isUsingFrontCamera`** variable changes. The camera preview is automatically updated to reflect the new camera selection.

It's also possible to provide your users with more control over the camera functionality. So, let's talk about **`CameraControls`**.

CameraControls

A comprehensive camera experience isn't just about capturing or analyzing an image. It's also about control. With **`CameraControls`**, developers gain access to an array of functions that allow them to manipulate the camera feed. From zooming into a subject and adjusting focus for that crystal-clear shot to toggling the torch for those night-time snaps, **`CameraControls`** ensures users always get the perfect shot.

Here is an example of how to use **`CameraControls`** to zoom, adjust focus, and toggle the torch, starting with the first part of the code:

```
@Composable
fun CameraControlsExample() {
    val cameraProvider = rememberCameraProvider()
    val preview = remember { Preview.Builder().build() }
    val zoomLevel = remember { mutableStateOf(1.0f) }
    val focusPoint = remember { mutableStateOf(0.5f, 0.5f) }
    val isTorchEnabled = remember { mutableStateOf(false) }
    val imageAnalyzer = ImageAnalysis.Builder()
        .setAnalyzer { image ->
            // Process the image data here
        }
        .build()
```

In the preceding code, we are defining the **`rememberCameraProvider`** function, which is used to retrieve the camera provider instance. It manages the camera's life cycle and provides access to camera controls. Then, **`Preview.Builder()`** is used to create a **`Preview`** instance, which defines the surface on which the live camera feed will be displayed.

Three **`mutableStateOf`** variables are used to store the state of the zoom level, focus point, and torch status:

- **`zoomLevel`**: This stores the current zoom level, ranging from 1.0f (no zoom) to 5.0f (maximum zoom)
- **`focusPoint`**: This stores the current focus point, represented as a pair of coordinates (*x*, *y*) within the preview frame
- **`isTorchEnabled`**: This stores the current torch status, indicating whether the torch is enabled or disabled

Let's continue with the next part of the code:

```
    LaunchedEffect(cameraProvider) {
        val cameraControl =
            cameraProvider.getCameraControl(preview)
        cameraControl.setZoomRatio(zoomLevel.value)
        cameraControl.setFocusPoint(focusPoint.value)
        cameraControl.enableTorch(isTorchEnabled.value)
```

```
        val useCaseBinding = UseCaseBinding.Builder()
            .addUseCases(preview, imageAnalyzer)
            .build()
        val camera =
            cameraProvider.bindToLifecycle(useCaseBinding)
        camera.close()
    }
```

Here, the **`cameraControl.getCameraControl(preview)`** method retrieves the **`CameraControl`** instance associated with the preview. This instance provides access to various camera controls:

- **`cameraControl.setZoomRatio(zoomLevel.value)`**: This control sets the zoom level using the value stored in the **`zoomLevel`** variable
- **`cameraControl.setFocusPoint(focusPoint.value)`**: This control sets the focus point using the coordinates stored in the **`focusPoint`** variable
- **`cameraControl.enableTorch(isTorchEnabled.value)`**: This control enables or disables the torch based on the value stored in the **`isTorchEnabled`** variable

Now, let's move on to the last chunk of code:

```
    Box(modifier = Modifier.fillMaxSize()) {
        Preview(preview)
        Column {
            Slider(
                value = zoomLevel.value,
                onValueChange = { zoomLevel.value = it },
                valueRange = 1.0f..5.0f,
                steps = 10
            ) {
                Text("Zoom")
            }
            Button(onClick = {
                val newFocusPoint = if (focusPoint.value ==
                0.5f) {
                    0.1f to 0.1f
                } else {
                    0.5f to 0.5f
                }
                focusPoint.value = newFocusPoint
                cameraControl.setFocusPoint(newFocusPoint)
            }) {
                Text("Adjust Focus")
            }
            Button(onClick = {
                isTorchEnabled.value =
                    !isTorchEnabled.value
                cameraControl.enableTorch(
                    isTorchEnabled.value)
            }) {
                Text("Toggle Torch")
            }
        }
```

```
    }
}
```

In this last code block, the controls are configured and used within the **`Column`** layout:

- A **`Slider`** component is used to adjust the zoom level. The **`valueRange`** property defines the range of zoom levels (1.0f to 5.0f), and the **`onValueChange`** callback updates the **`zoomLevel`** variable with the selected zoom level.
- A **`Button`** component triggers a change in the focus point. When clicked, it updates the **`focusPoint`** variable between two predefined locations (0.5f to 0.5f and 0.1f to 0.1f).
- Another **`Button`** component toggles the torch status. When clicked, it updates the **`isTorchEnabled`** variable and calls **`cameraControl.enableTorch`** to set the torch accordingly.

In conclusion, CameraX provides a robust and versatile platform for developing high-quality camera applications on Android. It offers a simplified API, streamlined use cases, and a comprehensive set of features, making it an ideal choice for building modern camera-centric apps. Now, we are ready to use it in our app.

Integrating CameraX into our Packtagram app

Now that we know more about CameraX, let's start integrating it into our app. First, we will need to deal with the camera permissions, providing a way for the user to accept them. Then, we will set up our camera preview and add the camera capture functionality to our code.

Setting up the permissions checker with Accompanist

There are several ways to check if the camera permissions have been granted, and if not, to request them: we could do this manually or use a library. In this case, we will use the Accompanist library, as we introduced at the beginning of this chapter.

Before requesting any permission at runtime, it's fundamental to declare the same permissions in the app's **`AndroidManifest.xml`** file. This declaration informs the Android operating system of the app's intentions. For the camera permission, you need to add the following line within the **`<manifest>`** tag:

```
<uses-permission android:name="android.permission.CAMERA" />
```

While the manifest informs the system of the app's needs, runtime permissions are about seeking the user's explicit consent. Ensure you always

have both in place when accessing protected features or user data.

Now, let's go into the permissions checker code. Our aim here is to create a reusable composable function that can handle the camera permission elegantly. It should be able to request the permission, handle user decisions, and, if necessary, explain why the app needs this permission.

First, we need to import the required libraries:

```
import com.google.accompanist.permissions.ExperimentalPermissionsApi
import com.google.accompanist.permissions.PermissionState
import com.google.accompanist.permissions.rememberPermissionState
@OptIn(ExperimentalPermissionsApi::class)
@Composable
fun CameraPermissionRequester(onPermissionGranted: () -> Unit) {
    // ... code ...
}
```

Here, the **`@OptIn`** annotation indicates that we're using an experimental API from the Accompanist permissions library.

Now, inside **`CameraPermissionRequester`**, we need to add the following:

```
val cameraPermissionState = rememberPermissionState(Manifest.permission.CAMERA)
```

Here, **`rememberPermissionState`** is a helper function that recalls the current state of the camera permission. It provides information such as whether the permission is granted, if we've already asked the user, or if we should show a rationale.

With the permission state in hand, we can create a UI flow that responds to this state:

- **Permission granted**: If permission is already granted, the user can directly proceed to use the camera.
- **Show rationale**: Sometimes, if a user denies a certain permission, it's helpful to explain why the app needs that permission. This is where the rationale comes into play.
- **Permission not yet requested**: If the app hasn't asked for the permission yet, we want to provide a button to initiate the request.
- **Permission denied without rationale**: In some cases, users deny permissions and opt not to be asked again. It's good practice to guide them to the app settings if they change their mind.

Let's learn how to handle all these possible flows. First, we will create a new composable called **`CameraPermissionRequester`**. The **`onPermissionGranted`** callback is provided to handle the scenario when the camera permission has been granted:

```
@OptIn(ExperimentalPermissionsApi::class)
@Composable
```

```
fun CameraPermissionRequester(onPermissionGranted:
@Composable () -> Unit) {
```

Next, we will retrieve **`cameraPermissionState`**:

```
    // Camera permission state
    val cameraPermissionState = rememberPermissionState(
        android.Manifest.permission.CAMERA
    )
```

The **`rememberPermissionState(permission)`** function retrieves the current state of the specified permission. In this case, we're checking the status of the **`CAMERA`** permission, which is necessary for accessing the device's camera. The result is stored in the **`cameraPermissionState`** variable.

Now, let's evaluate the different values it could have:

```
    if (cameraPermissionState.status.isGranted) {
        OnPermissionGranted.invoke()
```

In the previous code block, we are starting to evaluate the **`status.isGranted`** property of the **`cameraPermissionState`** object, which indicates whether the permission has been granted. If it's true, it means the permission is available, and we can call the **`onPermissionGranted`** callback to proceed with using the camera features.

If it is false, this means that the permission hasn't been granted, so we will have to communicate that situation to the user and give them the option to grant it:

```
    } else {
                Surface(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(16.dp)
                        .padding(top = 24.dp),
                    color =
                      MaterialTheme.colorScheme.background,
        ) {
            Column(
                modifier = Modifier.padding(16.dp),
                verticalArrangement =
                    Arrangement.spacedBy(12.dp),
                horizontalAlignment =
                    Alignment.CenterHorizontally
            ) {
                val textToShow = if
                (cameraPermissionState.shouldShowRationale)
                {
                    "The camera and record audio are
                     important for this app. Please grant
                     the permissions."
                } else {
```

```
                    "Camera permission is required for this
                     feature to be available. Please grant
                     the permission."
                }
                Text(
                    text = textToShow,
                    style =
                    MaterialTheme.typography.bodyLarge.copy
                    (
                        fontSize = 16.sp,
                        fontWeight = FontWeight.Medium
                    ),
                    color =
                    MaterialTheme.colorScheme.onBackground
                )
                Button(
                    onClick = { cameraPermissionState
                        .launchMultiplePermissionRequest()
                        },
                    colors = ButtonDefaults.buttonColors(
                        containerColor =
                        MaterialTheme.colorScheme.primary,
                        contentColor =
                        MaterialTheme.colorScheme.onPrimary
                    ),
                    contentPadding = PaddingValues(12.dp)
                ) {
                    Text("Request Permission",
                        fontSize = 14.sp,
                            fontWeight = FontWeight.Bold)
                }
            }
        }
    }
}
```

In the previous code block, we're displaying a message explaining the need for the permission and providing a **`Button`** component to initiate the permission request process. The **`onClick`** handler of the button triggers the **`launchPermissionRequest()`** method of the **`cameraPermissionState`** object, which prompts the user to grant the permission.

The **`launchPermissionRequest()`** method opens a system dialogue requesting the user to grant the **`CAMERA`** permission. The dialogue provides clear instructions and explains the reasons why the permission is required.

If we run this code now, we should see the two screens. First, we will see our screen with the message to request the permissions (left). Once we click **Request permission**, we will see the system prompt to accept the permission (right):

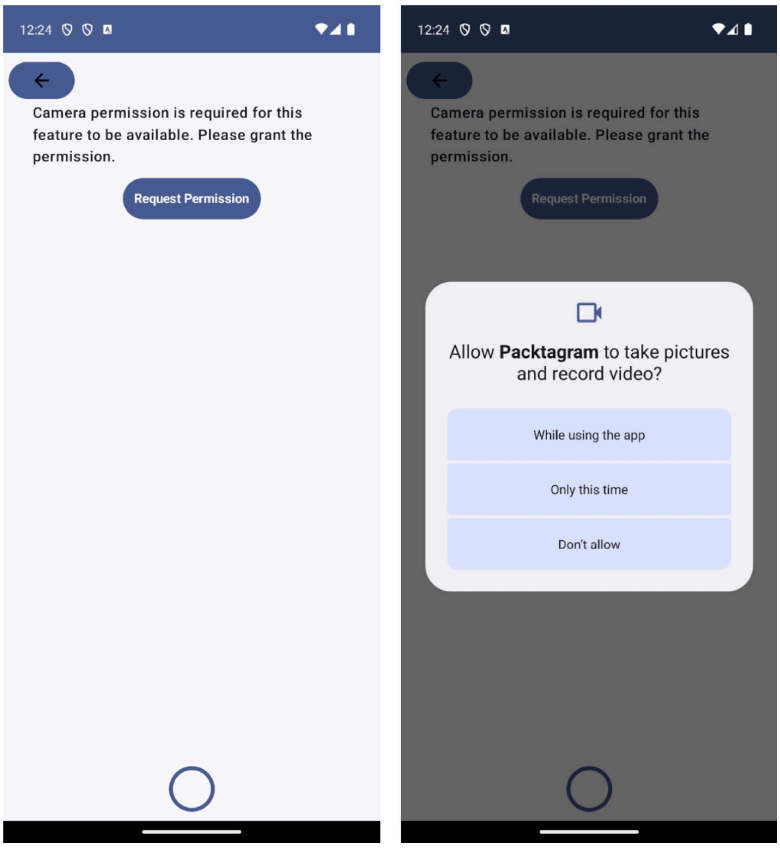

Figure 5.1: Camera permission requested in our app (left) and system prompt to grant capture and record permissions (right)

Once the permissions have been granted, **`CameraPreview`** can start working. We will use the **`onPermissionGranted`** callback to show it.

Creating our own CameraPreview

The following **`CameraPreview`** composable function is designed to elegantly integrate CameraX into the Jetpack Compose ecosystem. At the time of writing, there is not an official composable implementation for the CameraX preview, so we will use **`AndroidView`**:

```
@Composable
@Composable
fun CameraPreview(cameraController:
LifecycleCameraController, modifier: Modifier = Modifier) {
    AndroidView(
        factory = { context ->
            PreviewView(context).apply {
                implementationMode =
                  PreviewView.ImplementationMode.COMPATIBLE
            }
        },
        modifier = modifier,
        update = { previewView ->
            previewView.controller = cameraController
        }
```

```
    )
}
```

This composable function takes two parameters: **`cameraController`**, which is an instance of **`LifecycleCameraController`** to control the camera, and an optional modifier, which is used to specify layout options.

Inside the function, an **`AndroidView`** composable is used to bridge traditional Android views with the Jetpack Compose UI framework. The factory parameter of **`AndroidView`** is a Lambda that provides context and returns a **`PreviewView`** object. The **`PreviewView`** object is a standard Android view that's used to display the camera feed. It is configured with **`implementationMode`** set to **`COMPATIBLE`** to ensure compatibility with different devices and scenarios (one of the most relevant CameraX features).

The modifier parameter of **`AndroidView`** is set to the passed modifier to allow the layout to be customized. The **`update`** parameter is another Lambda that's called to perform updates on **`PreviewView`**. In this case, it assigns **`cameraController`** to the controller property of **`PreviewView`**, linking the camera preview to **`LifecycleCameraController`**.

Now, let's integrate the preview into our existing code. In the **`StoryContent`** composable, we will include the following code, where we expect to have the camera image:

```
CameraPermissionRequester {
    Box(contentAlignment = Alignment.BottomCenter,
    modifier = Modifier.fillMaxSize()) {
        CameraPreview(
            cameraController = cameraController,
            modifier = Modifier.fillMaxSize()
        )
    }
}
```

With that, we should be ready to use the camera! At this point, we've learned how to integrate **`CameraPreview`**, check the permissions, and show the camera image stream. Now, let's add the possibility of saving the photos!

Adding photo-saving functionality

The capture functionality is a staple for every app using the camera. We will need to add some logic to our existing code to handle the capture storage. Let's start with a use case (where we are going to put our domain logic) to store the captured image.

Creating the SaveCaptureUseCase

The primary responsibility of **`SaveCaptureUseCase`** will be to take a bitmap object (the format we will use for our photos) and save it as an image file in the device's gallery. Additionally, it will handle the different ap-

proaches based on the Android version as how media storage is accessed is different, depending on the version.

For example, we will need to obtain the URI (the route in the storage of the device) where we are going to store the image. If the user has a version of Android more recent than 9.0, the location will be different than in the previous versions. The following code block shows what the check to obtain the corresponding route will look like:

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
            MediaStore.Images.Media.getContentUri(
                MediaStore.VOLUME_EXTERNAL_PRIMARY)
        } else {
            MediaStore.Images.Media.EXTERNAL_CONTENT_URI
        }
}
```

Here, we are evaluating if the version is major or equal to Android 9.0 and obtaining the URI using **`MediaStore.Images.Media.getContentUri(MediaStore.VOLUME_EXTERNAL_PRIMARY)`**. If the version doesn't meet those requirements, we obtain the URI from **`MediaStore.Images.Media.EXTERNAL_CONTENT_URI`**. We should take all these different cases into account so that our use case handles the different Android versions properly.

Now, let's create the **`SaveCaptureUse`** class:

```
class SaveCaptureUseCase(private val context: Context) {
}
```

Then, we can create the main function of this use case, **`save()`**, which will take care of saving the capture:

```
    suspend fun save(capturePhotoBitmap: Bitmap):
    Result<Uri> = withContext(Dispatchers.IO) {
        val resolver: ContentResolver =
            context.applicationContext.contentResolver
        val imageCollection = getImageCollectionUri()
        val nowTimestamp = System.currentTimeMillis()
        val imageContentValues =
            createContentValues(nowTimestamp)
        val imageMediaStoreUri: Uri? =
            resolver.insert(imageCollection,
                imageContentValues)
        return@withContext imageMediaStoreUri?.let { uri ->
            saveBitmapToUri(resolver, uri,
                capturePhotoBitmap, imageContentValues)
        } ?: Result.failure(Exception("Couldn't create file
                                       for gallery"))
    }
```

In this code block, we are starting to create the save function. As it is marked as a **`suspend`** function, the save function is designed to be called

within a coroutine context. It uses **`withContext(Dispatchers.IO)`** to ensure that all I/O operations are performed on a background thread. This is crucial for maintaining UI responsiveness as I/O operations can be time-consuming.

Next, we are declaring **`ContextResolver`**. This resolver is used to interact with **`MediaStore`**, which is Android's central repository for media files.

Then, the function will call **`getImageCollectionUri()`**, a helper function that provides the appropriate URI for **`MediaStore`** based on the Android version. This URI is where the image will be saved. We will implement this function next.

After, the current system time (**`nowTimestamp`**) is captured, and **`create-ContentValues (nowTimestamp)`** is invoked to prepare the metadata for the image. This metadata, which is stored in a **`ContentValues`** object, includes details such as the image's display name, **`MIME`** type, and timestamps.

The function then attempts to insert a new record into **`MediaStore`** using the resolved URI and the prepared metadata. The **`insert`** method returns a URI that points to the newly created record. If this operation is successful, a non-null URI is returned, representing the location of the new image record in **`MediaStore`**.

Finally, if the URI is not null, the **`saveBitmapToUri`** function is called with the resolver, the URI, the bitmap to be saved, and the image metadata. This function handles the actual process of writing the bitmap data to the location pointed to by the URI. We will implement it soon.

Regarding error handling, our **`save`** function uses Kotlin's **`Result`** class for structured error handling. If the insertion into the MediaStore is successful and the bitmap is saved correctly, the function returns **`Result.success(Unit)`**. If there is a failure at any point (for example, the URI is null, indicating that the insertion failed), the function returns **`Result.failure`**, encapsulating an exception with an appropriate error message.

Now, let's implement the **`getImageCollectionUri()`** function, which will return the correct URI based on the Android version:

```
    private fun getImageCollectionUri(): Uri =
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q)
        {
            MediaStore.Images.Media.getContentUri(
                MediaStore.VOLUME_EXTERNAL_PRIMARY)
        } else {
            MediaStore.Images.Media.EXTERNAL_CONTENT_URI
        }
```

Then, we can create the **`createContentValues`** function:

```
private fun createContentValues(timestamp: Long):
ContentValues = ContentValues().apply {
        put(MediaStore.Images.Media.DISPLAY_NAME,
            "$FILE_NAME_PREFIX${System.currentTimeMillis()}
                .jpg")
        put(MediaStore.Images.Media.MIME_TYPE, "image/jpg")
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q)
        {
            put(MediaStore.MediaColumns.DATE_TAKEN,
                timestamp)
            put(MediaStore.MediaColumns.RELATIVE_PATH,
                "${Environment.DIRECTORY_DCIM}/Packtagram")
            put(MediaStore.MediaColumns.IS_PENDING, 1)
        }
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.R)
        {
            put(MediaStore.Images.Media.DATE_ADDED,
                timestamp)
            put(MediaStore.Images.Media.DATE_MODIFIED,
                timestamp)
            put(MediaStore.Images.Media.AUTHOR,
                AUTHOR_NAME)
            put(MediaStore.Images.Media.DESCRIPTION,
                DESCRIPTION)
        }
    }
```

The **`createContentValues`** function is designed to prepare the metadata for an image file before it is saved to the device's gallery via **`MediaStore`**. This method is pivotal in ensuring that the saved image has the correct and necessary information associated with it. So, let's break down its functionality:

- First, the function initiates a **`ContentValues`** object. Here, **`ContentValues`** is a key-value pair that's used in Android to store a set of values that **`ContentResolver`** can process. It is commonly used for passing data to Android's content providers
- Next, the display name of the image in **`MediaStore`** is set. We will use a predefined **`FILE_NAME_PREFIX`** constant and append the current time-stamp to it, followed by the **`.jpg`** extension, ensuring each saved image has a unique name.
- Then, the **`MIME`** type of the image is set to **`image/jpg`**. This information is used by **`MediaStore`** and other apps to understand the file format of the image.
- We have to store it differently, depending on the Android version of the device:
  - For Android Q (API Level 29) and above, we must do the following:
    - We need to add the timestamp of when the image is being stored and use the **`MediaStore.MediaColumns.DATE_TAKEN`** key.
    - We must use the **`createContentValues`** function to specify a relative path for the image file, pointing to a directory within the **Digital Camera Images** (**DCIM**) folder using **`put(MediaStore.MediaColumns.RELATIVE_PATH,`**

"${Environment.DIRECTORY_DCIM}/Packtagram"). This helps in organizing the saved images in a specific subdirectory, making them easier to locate.

- We need to update the **ContentValues** instance and set **IS_PENDING** to **1** (true), indicating that file creation is in progress. This is a way to inform the system and other apps that the file is not yet fully written and should not be accessed until the status is reverted.

- For Android R (API Level 30) and above, our function should add more metadata, including the date added, date modified, author name, and a description. This is part of the enhanced metadata management in newer Android versions, allowing for more detailed information to be stored with media files.

Now that we are handling the URI that's needed to store the file, as well as the values and metadata needed to create the file, let's proceed to do the saving itself. To do so, we will create a new private function called **saveBitmapToUri**, as follows:

```
    private fun saveBitmapToUri(
        resolver: ContentResolver,
        uri: Uri,
        bitmap: Bitmap,
        contentValues: ContentValues
    ): Result<Uri> = kotlin.runCatching {
       resolver.openOutputStream(uri).use { outputStream ->
           checkNotNull(outputStream) { "Couldn't create
               file for gallery, MediaStore output stream
                   is null»}`
           bitmap.compress(Bitmap.CompressFormat.JPEG,
               IMAGE_QUALITY, outputStream)
        }
```

The function starts by attempting to open **OutputStream** for the given URI. This stream is where the bitmap data will be written. Here, **Resolver.openOutputStream(uri)** is used to obtain the stream, and the **use** block ensures that this stream is closed properly after its operations, following the best practices in resource management.

Inside the **use** block, the function checks if **outputStream** is not **null**, throwing an exception with a descriptive message if it is. If the stream is valid, the bitmap is compressed and written to this stream. The compression format is set to JPEG, and the quality is determined by the **IMAGE_QUALITY** constant.

Now, if the image is saved successfully, we have to update and return the result. If something has failed, we have to return an error:

```
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q)
        {
            contentValues.clear()
```

```
            contentValues.put(
                MediaStore.MediaColumns.IS_PENDING, 0)
            resolver.update(uri, contentValues, null, null)
        }
        return Result.success(Unit)
    }.getOrElse { exception ->
        exception.message?.let(::println)
        resolver.delete(uri, null, null)
        return Result.failure(exception)
    }
}
```

For devices running Android Q (API level 29) or higher, after the image is saved, the function updates the **`MediaStore`** entry to indicate that the image is no longer pending. This is done by clearing the existing **`contentValues`**, setting **`IS_PENDING`** to **`0`** (false), and then updating the **`MediaStore`** entry with these new values. This step is crucial for making the image available to the user and other applications.

The entire operation is wrapped in a **`runCatching`** block, which is a Kotlin construct that's used for simplified exception handling. This block captures any exceptions that occur during the **`OutputStream`** operation or **`MediaStore`** update. If an exception occurs, it is logged, and the function attempts to delete the possibly corrupted or incomplete file from **`MediaStore`**. This cleanup is essential to prevent cluttering the storage with unusable files.

The function returns **`Result<Uri>`**, indicating the success or failure of the operation. In case of success, **`Result.success(uri)`** is returned. In case of an exception, **`Result.failure(exception)`** is returned, encapsulating the exception details.

The only thing left will be to add the parameters that will be used during the development of these classes. For simplicity, we will add them as constants, but they could also be provided to the class:

```
companion object {
    private const val IMAGE_QUALITY = 100
    private const val FILE_NAME_PREFIX = "YourImageName"
    private const val AUTHOR_NAME = "Your Name"
    private const val DESCRIPTION = "Your description"
}
```

The next step is to integrate this use case in **`StoryEditorViewModel`**.

Integrating SaveCaptureUseCase in StoryEditorViewModel

Here, we need to create a new property and function in **`StoryEditorViewModel`** to store the captured image:

```
class StoryEditorViewModel(
```

```
    private val saveCaptureUseCase: SaveCaptureUseCase
): ViewModel() {
    private val _isEditing = MutableStateFlow(false)
    val isEditing: StateFlow<Boolean> = _isEditing
    private val _imageCaptured: MutableStateFlow<Uri> =
        MutableStateFlow(Uri.EMPTY)
    val imageCaptured: StateFlow<Uri> = _imageCaptured
    fun storePhotoInGallery(bitmap: Bitmap) {
        viewModelScope.launch {
            val imageUri =
                saveCaptureUseCase.save(bitmap).getOrNull()
            if (imageUri != null) {
                _imageCaptured.value = imageUri
                _isEditing.value = true
            }
    }
}
```

In this **`storePhotoInGallery`** function, we are just launching a coroutine to call the **`saveCaptureUseCase.save`** method. Then, once we've obtained the URI, we check if it is not **`null`** and update the **`imageCaptured`** property.

Finally, we are ready to add this functionality to the UI.

Adding the capture functionality to StoryContent

To add the capture functionality to **`StoryContent`**, we need to add a Lambda to the **`StoryContent`** composable so that whenever we use **`StoryContent`**, capture handling will be delegated. For example, in our case, we will call the already implemented **`storePhotoInGallery`** function from **`StoryEditorViewModel`**:

```
@Composable
fun StoryContent(
    isEditing: Boolean = false,
    onImageCaptured: (Bitmap) -> Any,
    modifier: Modifier = Modifier,
) { ... }
```

Next, let's integrate the code that's needed to take the capture from our camera:

```
fun capturePhoto(
        context: Context,
        cameraController: LifecycleCameraController,
        onPhotoCaptured: (Bitmap) -> Unit,
        onError: (Exception) -> Unit
    ) {
```

The parameters we are using in the previous code block are as follows:

- **`context`**: The Android context that we will use to obtain **`MainExecutor`**.
- **`cameraController`**: A **`LifecycleCameraController`** object from **`CameraX`**, which controls the camera's life cycle and operations.

- **`onPhotoCaptured`**: The callback function that will be invoked when a photo is successfully captured and processed. It accepts a **`Bitmap`** as its parameter.
- **`onError`**: A callback function to handle any errors that occur during the photo capture process.

Let's continue by defining the necessary properties:

```
val mainExecutor: Executor =
ContextCompat.getMainExecutor(context)
```

Here, we will retrieve **`MainExecutor`**. This executor is used to run tasks on the Android main thread, which is essential for UI updates and certain CameraX operations. It is needed for **`CameraController`**.

Next, we will execute the take picture action:

```
        cameraController.takePicture(mainExecutor,
        @ExperimentalGetImage object :
        ImageCapture.OnImageCapturedCallback() {
            override fun onCaptureSuccess(image:
            ImageProxy) {
                try {
                    CoroutineScope(Dispatchers.IO).launch {
                        val correctedBitmap: Bitmap? =
                            image
                                ?.image
                                ?.toBitmap()
                                ?.rotateBitmap(image
                                    .imageInfo
                                    .rotationDegrees)
                        correctedBitmap?.let {
                            withContext(Dispatchers.Main) {
                                onPhotoCaptured(
                                    correctedBitmap)
                            }
                        }
                        image.close()
                    }
                } catch (e: Exception) {
                    onError(e)
                } finally {
                    image.close()
                }
            }
            override fun onError(exception:
            ImageCaptureException) {
                Log.e("CameraContent", "Error capturing
                    image", exception)
                onError(exception)
            }
        })
    }
```

Here, we call the **`cameraController.takePicture`** method. We will need to provide it with the executor and an **`ImageCapture.OnImageCapturedCallback`** class. This class provides callback methods for when an image is successfully captured or when an error occurs.

In the case of success, we will switch to the **I/O dispatcher** so that we can process the ImageProxy transformation into a bitmap in the background. Once it's been transformed, we call the **`onPhotoCaptured`** Lambda from the main dispatcher. Alternatively, if there is any error, we will receive them via the **`onError(exception: ImageCaptureException)`** callback. Then, we will pass the error to the **`onError`** callback function, which we received as the parameter of the **`capturePhoto()`** function.

Now, let's link the capture functionality with our UI. We already have a button for doing the capture in our **`StoryContent`** composable, **`OutlinedButton`**, so let's see how we can call this capture function from it:

```
OutlinedButton(
                    onClick = { capturePhoto(
                        context = localContext,
                        cameraController =
                            cameraController,
                        onPhotoCaptured = {
                            onImageCaptured(it) },
                        onError = { /* Show error */ }
                            )
                    },
                    modifier = Modifier.size(50.dp),
                    shape = CircleShape,
                    border = BorderStroke(4.dp,
                        MaterialTheme.colorScheme.primary),
                    contentPadding = PaddingValues(0.dp),
                    colors =
                        ButtonDefaults.outlinedButtonColors
                            (contentColor =
                                MaterialTheme.colorScheme
                                    .primary)
                ) {
                }
```

As we can see, we are calling the **`capturePhoto`** function from the **`onClick`** button.

With this, we are ready to capture our photos:

Figure 5.2: Image preview with the capture button

With that, we have created a use case so that we can store our photos and link the functionality with our already existing UI. Our users can also capture and store their photos. Next, let's see if we can enable them so that we can edit some aspects of them.

Adding photo-editing functionalities

There are multiple operations that we can enable for the user to edit and modify their images: we can allow them to crop, resize, and rotate the im-

age, as well as adjust the brightness and contrast, apply filters, or add text overlays.

As part of this chapter, we are going to implement two operations: a black-and-white filter and a text overlay.

Adding filters

Creating filters over an existing image is as easy as modifying the values of the bitmap that contains the image. There are several well-known filters, such as sepia, vintage, and black and white. As an example, we are going to implement the black and white filter, like so:

```
@Composable
fun BlackAndWhiteFilter(
    imageUri: Uri,
    modifier: Modifier = Modifier
) {
    var isBlackAndWhiteEnabled by remember {
    mutableStateOf(false) }
    val localContext = LocalContext.current
    Box(modifier = modifier.fillMaxSize()) {
        Canvas(modifier = Modifier.fillMaxSize()) {
            getBitmapFromUri(localContext, imageUri)?.let {
                val imageBitMap = it.asImageBitmap()
                val colorFilter = if
                (isBlackAndWhiteEnabled) {
                    val colorMatrix = ColorMatrix().apply {
                        setToSaturation(0f) }
                    ColorFilter.colorMatrix(colorMatrix)
                } else {
                    null
                }
                val (offsetX, offsetY) =
                    getCanvasImageOffset(imageBitMap)
                val scaleFactor =
                    getCanvasImageScale(imageBitMap)
                with(drawContext.canvas) {
                    save()
                    translate(offsetX, offsetY)
                    scale(scaleFactor, scaleFactor)
                    drawImage(
                        image = imageBitMap,
                        topLeft =
                            androidx.compose.ui.geometry
                                .Offset.Zero,
                        colorFilter = colorFilter
                    )
                    restore()
                }
            }
        }
        Button(
            onClick = { isBlackAndWhiteEnabled =
                !isBlackAndWhiteEnabled },
            modifier = Modifier.padding(16.dp)
```

```
        ) {
            Text("Apply Black and White Filter")
        }
    }
}
```

This function starts by accepting **`imageUri`**, which is the URI representing the image to be displayed, and an optional modifier parameter to customize the layout.

Within the function, a state variable called **`isBlackAndWhiteEnabled`** is declared using **`remember`** and **`mutableStateOf`**, which tracks whether the black-and-white filter is applied. Here, **`LocalContext.current`** provides the context needed to load the image from the URI.

A **`Box`** composable is used to contain the entire layout, ensuring that the content fills the available space. Inside **`Box`**, a **`Canvas`** composable is used to draw the image. The **`Canvas`** modifier is set to fill the available size.

The **`Canvas`** composable uses the **`getBitmapFromUri`** function to load the image as a **`Bitmap`**, which is then converted into **`ImageBitmap`** using the **`asImageBitmap`** extension function. If the **`isBlackAndWhiteEnabled`** state is true, a **`ColorMatrix`** value with zero saturation is applied to create a black-and-white **`ColorFilter`**. Otherwise, no color filter is applied.

The **`getCanvasImageOffset`** and **`getCanvasImageScale`** functions are used to calculate the offset and scale factor needed to center and scale the image within the canvas. The **`with(drawContext.canvas)`** block is used to draw the image. Within this block, **`save`** and **`restore`** are called to save and restore the canvas state, ensuring that transformations do not affect subsequent drawing operations. The **`translate`** function applies the calculated offsets, and the **`scale`** function applies the scale factor, to fill the entire **`Canvas`** with the image. Finally, the **`drawImage`** function draws the image on the canvas with the optional color filter.

Below **`Canvas`**, a **`Button`** composable is placed within **`Box`**. This button is used to toggle the **`isBlackAndWhiteEnabled`** state when clicked. The button's **`onClick`** Lambda updates the state variable, and the button's text is set to **Apply Black and White Filter**. The modifier parameter for the button includes padding to ensure it is not placed at the edge of the screen.

Now that we have built our first filter, let's learn how to implement text overlays.

Adding a text overlay

Adding a text overlay is a typical image editing functionality that allows us to tag other people, add a hashtag to an image, or add an accompanying written message. Let's see how we can offer our users this functionality.

First, we are going to create a composable that contains the state of the **`Text`** and **`Image`** components. This state will update as the user updates the text. Here's the code:

```
@Composable
fun ImageWithTextOverlay(capturedBitmap: Bitmap) {
    var textOverlay = remember { mutableStateOf("Add your
        text here") }
    var showTextField = remember { mutableStateOf(false) }
    Box(modifier = Modifier.fillMaxSize()) {
        Image(
            bitmap = capturedBitmap.asImageBitmap(),
            contentDescription = "Captured Image",
            modifier = Modifier.matchParentSize()
        )
        if (showTextField) {
            TextField(
                value = textOverlay,
                onValueChange = { textOverlay = it },
                modifier = Modifier
                    .align(Alignment.Center)
                    .padding(16.dp)
            )
        }
        Text(
            text = textOverlay,
            color = Color.White,
            fontSize = 24.sp,
            modifier = Modifier.align(Alignment.Center)
        )
        FloatingActionButton(
            onClick = { showTextField = !showTextField },
            modifier = Modifier
                .align(Alignment.BottomEnd)
                .padding(16.dp)
        ) {
            Icon(Icons.Default.Edit, contentDescription =
                "Edit Text")
        }
    }
}
```

This example defines a composable function called **`ImageWithTextOverlay`**. It accepts a bitmap object named **`captured-Bitmap`**, which represents the captured image that will be displayed with a text overlay.

The function starts by defining two pieces of state:

- First, we have **`textOverlay`**, which holds the text that will be displayed over the image. It's initially set to a default value of **Add your text here**.
- Then, we have a **`showTextField`** Boolean, which determines whether the text editing field (**`TextField`**) is visible or not. It's initially set to **`false`**.

Within the function, we use a **`Box`** composable as a container. The **`Box`** composable allows us to layer its child components, and we set its size to fill the maximum available space. This creates an area where we can overlay text on top of an image.

The first child of the **`Box`** composable is an **`Image`** composable, which is responsible for displaying the captured photo. The photo is passed to this function as a bitmap, and we ensure that it fills the entire parent container, ensuring that the image takes up the whole screen space available.

Next, we check the state of **`showTextField`**. If it's **`true`**, we display **`TextField`** in the center of the screen. This **`TextField`** allows the user to input or edit the text that will be overlaid on the image. As the user types, the text in **`textOverlay`** is updated in real time thanks to the two-way binding provided by Jetpack Compose.

Regardless of the state of **`showTextField`**, we always display a **`Text`** composable. This component is responsible for rendering the overlay text on top of the image. We style this text to be white and of a reasonable font size, ensuring it's visible against a variety of backgrounds.

Finally, at the bottom corner of the **`Box`** composable, we place **`FloatingActionButton`**. This button, when clicked, toggles the visibility of **`TextField`**, allowing the user to switch between viewing the overlaid text and editing it. The button is intuitively designed with an edit icon, signaling its purpose to the user.

Now, imagine that we want to allow the user to move the text whenever they want in the image. Let's implement some drag-and-drop magic. We will start by updating the **`ImageWithTextOverlay`** composable function:

```
@Composable
fun ImageWithTextOverlay(capturedBitmap: Bitmap) {
    var textOverlay = remember { mutableStateOf("Your text
        here") }
    var showTextField = remember { mutableStateOf(false) }
    var textPosition by remember {
        mutableStateOf(Offset.Zero) }
```

In this updated version of the **`ImageWithTextOverlay`** composable function, we've introduced an interactive feature that allows users to drag and position the text overlay anywhere on the image. To achieve this, we added a new state variable, **`textPosition`**, initialized to **`Offset.Zero`**. This state holds the current position of the text overlay on the screen. Now, we must create a new composable function, **`DraggableText`**, to handle the text display and its draggable functionality.

Let's add this **`DraggableText`** to our existing code:

```
    val imageModifier = Modifier.fillMaxSize()
```

```
    Box(modifier = Modifier.fillMaxSize()) {
        Image(
            bitmap = capturedBitmap.asImageBitmap(),
            contentDescription = "Captured Image",
            modifier = imageModifier
        )
        if (showTextField) {
            TextField(
                value = textOverlay,
                onValueChange = { textOverlay = it },
                modifier = Modifier
                    .align(Alignment.Center)
                    .padding(16.dp)
            )
        }
        DraggableText(
            text = textOverlay,
            position = textPosition,
            onPositionChange = { newPosition ->
                textPosition = newPosition },
            modifier = Modifier
                .offset { IntOffset(textPosition.x.toInt(),
                    textPosition.y.toInt()) }
                .align(Alignment.Center)
        )
        FloatingActionButton(
            onClick = { showTextField = !showTextField },
            modifier = Modifier
                .align(Alignment.BottomEnd)
                .padding(16.dp)
        ) {
            Icon(Icons.Default.Edit, contentDescription =
                "Edit Text")
        }
    }
}
```

Here, the existing functionality for editing the text through **`TextField`** is the same. The **`TextField`** field appears when the user wishes to edit the text, facilitated by a floating action button. This button toggles the visibility of **`TextField`**, allowing users to switch seamlessly between editing the text and adjusting its position.

Now, we are ready to create the **`DraggableText`** composable:

```
@Composable
fun DraggableText(
    text: String,
    position: Offset,
    onPositionChange: (Offset) -> Unit,
    modifier: Modifier = Modifier
) {
```

The **`DraggableText`** composable takes several parameters, including the text to display, its current position, and a callback function, **`onPositionChange`**, which updates this position. Within **`DraggableText`**, we utilize the draggable modifier on the **`Text`** composable. This modifier is pivotal as it

allows the text to be moved across the screen. As the user drags the text, the drag offset is updated, which, in turn, updates the **`textPosition`** state in the main **`ImageWithTextOverlay`** function.

Finally, define the variables that are needed and the **`Text`** composable to show the text:

```
var dragOffset = remember { mutableStateOf(position) }
Text(
    text = text,
    color = Color.White,
    fontSize = 24.sp,
    modifier = modifier
        .offset {
            IntOffset(dragOffset.value.x.roundToInt(),
                dragOffset.value.y.roundToInt()) }
        .pointerInput(Unit) {
            detectDragGestures { change, dragAmount ->
                change.consume()
                dragOffset.value =
                    Offset((dragOffset.value.x +
                        dragAmount.x),
                            (dragOffset.value.y +
                                dragAmount.y))
                onPositionChange(dragOffset.value)
            }
        }
        .background(
            color = Color.Black.copy(alpha = 0.5f),
            shape = RoundedCornerShape(8.dp)
        )
)
```

We begin by initializing a state to hold the current drag offset. This state will track the position of the text as it is dragged.

Next, we define the **`Text`** composable to display our draggable text. To control the positioning of the text, we use the offset modifier, which positions the text based on the current drag offset.

The **`pointerInput`** modifier allows us to handle drag gestures on the text element. Within the **`detectDragGestures`** block, we update the drag offset by adding the drag amount to the current offset each time the user drags the text. The gesture change is consumed to indicate that the drag event has been handled, and we call a function to handle any additional actions that are needed when the position changes.

And with that, here are the two filters we have created:

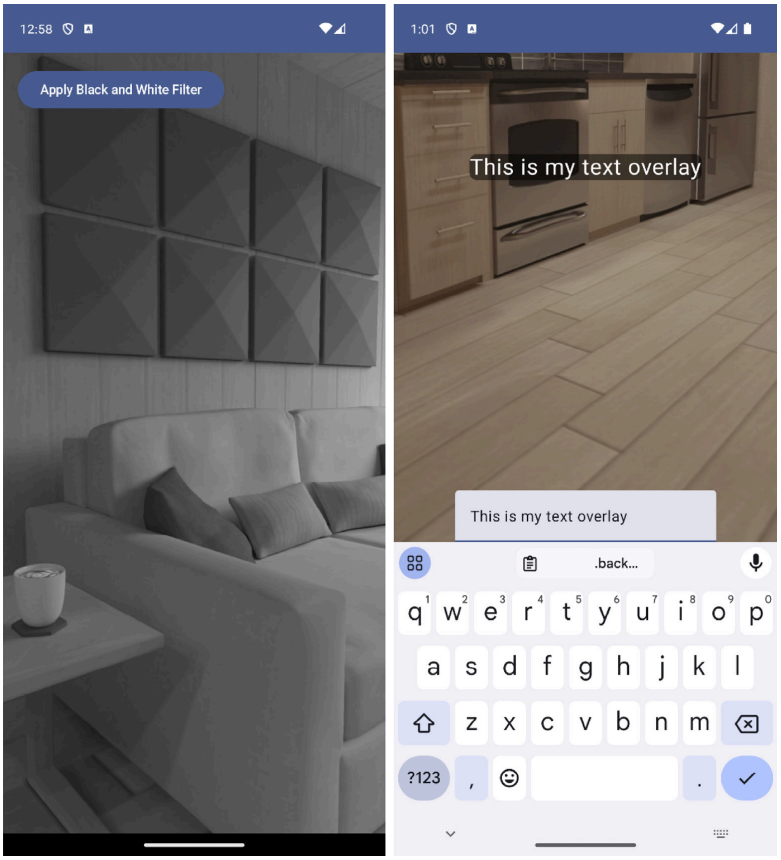

Figure 5.3: The black and white filter composable (left) and text overlay (right)

At this point, we have already implemented some cool features for our users, such as a black-and-white filter and the possibility to add a caption. So, why don't leverage the use of ML to build outstanding features? We'll look at this in the next section.

Using ML to categorize photos and generate hashtags

ML is a branch of **artificial intelligence** (**AI**) that focuses on building systems that can learn from and make decisions based on data. Unlike traditional software, which follows explicitly programmed instructions, ML algorithms use statistical techniques to enable computers to improve at tasks with experience. The fundamental premise of ML is to develop algorithms that can receive input data and use statistical analysis to predict or make decisions about some aspect of the data.

ML is a huge field that is outside the scope of this book, but we still can do interesting things using already-built libraries. For example, **ML Kit** is a powerful ML solution offered by Google for mobile developers that provides a suite of ready-to-use APIs for various ML tasks, both on-device and cloud-based. These functionalities are designed to be easily integrated into mobile applications, facilitating the use of ML without requiring deep expertise in the field. Here's an overview of the key functionalities offered by ML Kit:

- **Image labeling**: Identifies objects, locations, activities, animal species, products, and more within an image.
- **Text recognition**: Extracts text from images. This can be useful for **optical character recognition** (**OCR**) applications, such as scanning documents, business cards, or any printed or handwritten text.
- **Face detection**: Detects faces in an image, including key facial features such as eyes and nose, and characteristics such as smiles or head tilt. This is useful in applications such as photo tagging and facial recognition.
- **Barcode scanning**: Reads and scans barcodes and QR codes. It supports various formats, including UPC, EAN, Code 39, and others.
- **Object detection and tracking**: Identifies and tracks objects in an image or video stream. This feature is useful in scenarios such as real-time video analysis.

You can learn more about ML Kit's features at https://developers.google.com/ml-kit.

As an example, we are going to create the logic to identify and label elements in the photo that could be used in the future to categorize the images or create automatic hashtags. We will start by adding the corresponding dependencies to **`libs.versions.toml`**:

```
[versions]
...
ml-labeling = "17.0.5"
[libraries]
...
mlKitLabeling= { group = "com.google.mlkit", name = "image-labeling", version.ref="ml-labeling"}
```

Then, we will add these dependencies to the **`build.gradle`** file of the module. This is where we are creating this functionality (**`feature:stories`**):

```
    implementation(libs.mlKitLabeling)
```

Now, we can create the actual code. We are going to leverage the image analysis feature from CameraX and analyze the preview using MLKitLabeling before using the results to write them in over the image. To do this, we will create a new preview composable just for this feature:

```
@Composable
fun CameraPreviewWithImageLabeler(cameraController: LifecycleCameraController, modifier: Modifie
    val context = LocalContext.current
    var labels by remember {
        mutableStateOf<List<String>>(emptyList()) }
    val cameraProviderFuture = remember {
        ProcessCameraProvider.getInstance(context) }
    val previewView = remember { PreviewView(context) }
    val imageAnalysis = remember {
        ImageAnalysis.Builder()
```

```
                .setTargetResolution(Size(1280, 720))
                .setBackpressureStrategy(
                    ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)
                .build()
        }
        DisposableEffect(Unit) {
            val cameraProvider = cameraProviderFuture.get()
            val preview = Preview.Builder().build().also {
                it.setSurfaceProvider(
                    previewView.surfaceProvider)
            }
            val cameraSelector =
                CameraSelector.DEFAULT_BACK_CAMERA
            cameraProvider.bindToLifecycle(
                context as LifecycleOwner, cameraSelector,
                    preview, imageAnalysis)
            onDispose {
                cameraProvider.unbindAll()
            }
        }
        imageAnalysis.setAnalyzer(ContextCompat.getMainExecutor
        (context)) { imageProxy ->
            processImageProxyForLabeling(imageProxy) {
            detectedLabels ->
                labels = detectedLabels
            }
        }
        Box(modifier = modifier) {
            AndroidView(
                factory = { previewView },
                modifier = modifier
            )
            Canvas(modifier = Modifier.fillMaxSize()) {
                drawIntoCanvas { canvas ->
                    val paint = android.graphics.Paint().apply
                    {
                        color = android.graphics.Color.RED
                        textSize = 60f
                    }
                    labels.forEachIndexed { index, label ->
                        canvas.nativeCanvas.drawText(label,
                            10f, 100f + index * 70f, paint)
                    }
                }
            }
        }
    }
```

The start of this function is pretty similar to our already existing **`CameraPreview`** composable. After the camera provider is defined, an **`ImageAnalysis`** instance is configured with a target resolution of 1,280x720 pixels and a backpressure strategy set to **`STRATEGY_KEEP_ONLY_LATEST`** to process the latest image frame.

The **`imageAnalysis.setAnalyzer`** method sets an analyzer to process image frames using ML Kit's Image Labeler. The **`processImageProxyForLabeling`** function is called to process each image frame. The detected labels

are passed to a Lambda function that updates the **`labels`** state variable. We will see how to implement this function shortly.

In the end, the **`Box`** composable is used to overlay **`PreviewView`** and a **`Canvas`** composable. The **`Canvas`** composable is used to draw the detected labels on top of the camera preview. The **`drawIntoCanvas`** method accesses the native **`canvas`** for drawing. A **`Paint`** object is configured with a red color and a text size of 60 pixels. The **`forEachIndexed`** method iterates over the labels list, drawing each label at a specified position on the canvas.

Now, let's learn how we can implement the image analyzer:

```
@OptIn(ExperimentalGetImage::class)
private fun processImageProxyForLabeling(imageProxy:
ImageProxy, onLabelsDetected: (List<String>) -> Unit) {
    val mediaImage = imageProxy.image
    if (mediaImage != null) {
        val image = InputImage.fromMediaImage(mediaImage,
            imageProxy.imageInfo.rotationDegrees)
        val labeler =
        ImageLabeling.getClient(
            ImageLabelerOptions.DEFAULT_OPTIONS)
        labeler.process(image)
            .addOnSuccessListener { labels ->
                val labelNames = labels.map { it.text }
                onLabelsDetected(labelNames)
            }
            .addOnFailureListener { e ->
                e.printStackTrace()
            }
            .addOnCompleteListener {
                imageProxy.close()
            }
    }
}
```

This function takes the **`ImageProxy`** object and a callback function, **`onLabelsDetected`**, as parameters, where the callback function is invoked with a list of detected labels.

Within the function, **`mediaImage`** is extracted from the **`ImageProxy`** object. If **`mediaImage`** is not **`null`**, it is converted into **`InputImage`** using the **`InputImage.fromMediaImage`** method, which requires the media image and the rotation degrees from **`imageProxy`**.

An instance of the image labeler is obtained by calling **`ImageLabeling.getClient`** with **`ImageLabelerOptions.DEFAULT_OPTIONS`**. This sets up the labeler with default configuration options suitable for general-purpose image labeling.

The **`labeler.process`** method processes **`InputImage`** asynchronously. After, the processing outcome is handled by two listeners:

- In `addOnSuccessListener`, the function receives a list of labels if the processing is successful. Each label in this list represents an element identified in the image, accompanied by a confidence score. The function iterates through these labels, logging the identified element (`label.text`) and its confidence score (`label.confidence`). In future iterations, we could use this information to auto-create automatic overlays over the image or to inform the user of which could be the best hashtags for the image.
- In case of any failure during image processing, `addOnFailureListener` is invoked, which logs the error. This error handling is crucial for diagnosing issues that might occur during the ML process, such as problems with the input image or internal errors in the ML Kit processing pipeline.

Now, if we replace our `CameraPreview` composable with the `CameraPreviewImageLabeler` composable, we should see the results of the image analysis taking place:

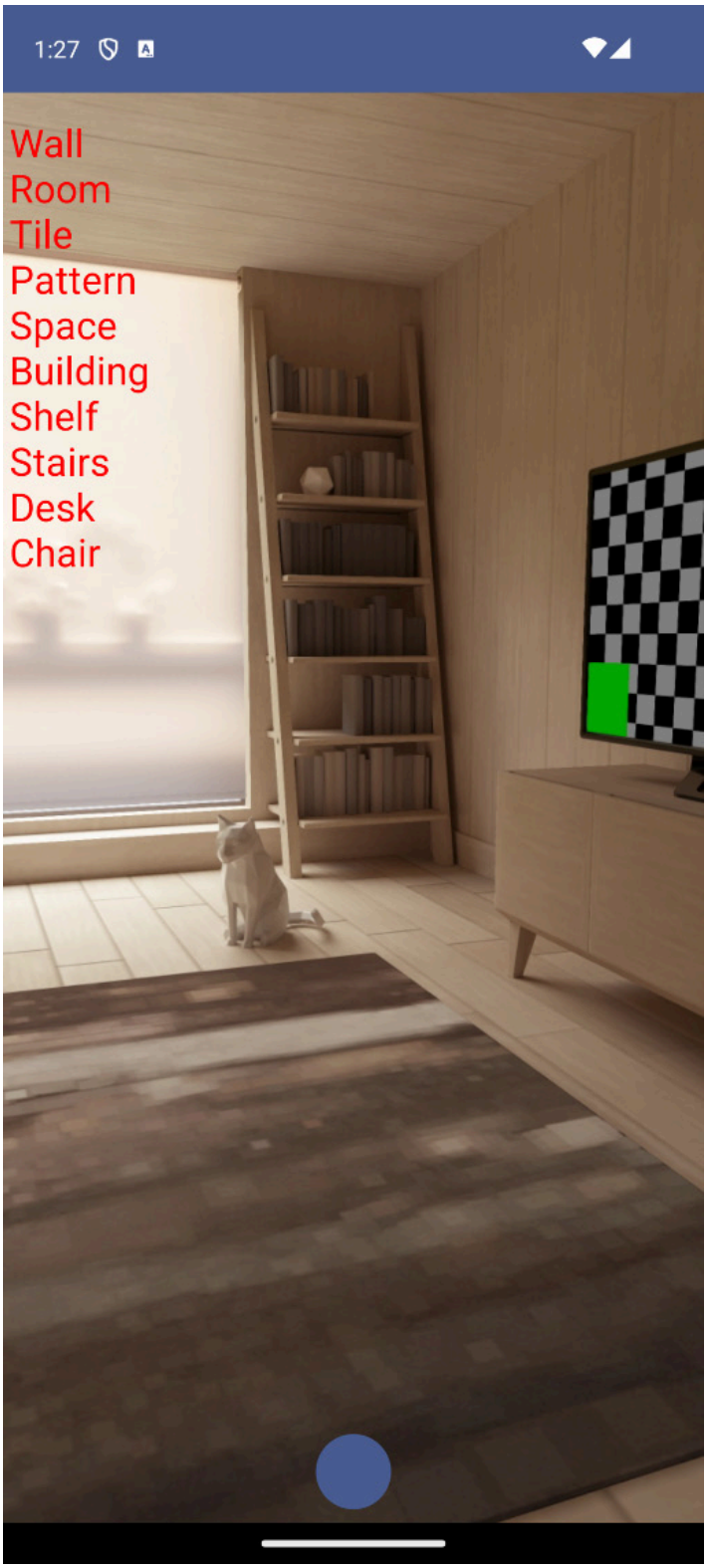

Figure 5.4: ML labeling taking place in the live preview

If you want to know more about what can be done with the ML Kit library, check out https://developer.android.com/ml.

Summary

In this chapter, we started by familiarizing ourselves with CameraX, a key component of the Android Jetpack suite. We learned how to set up

CameraX in our applications while enabling features such as live camera preview and image capture.

Moving on, we delved into the practical implementation of capturing images using CameraX. Additionally, we introduced basic image editing functionalities, guiding you through the process of creating a filter and adding a text overlay. These skills are pivotal in enhancing the interactivity and user experience of photography apps.

Finally, we unveiled the integration of Google's ML Kit, demonstrating how to add advanced ML capabilities to the app. We explored how to use ML Kit to identify elements in images, such as objects. This experience highlighted the practical application of these technologies in enhancing the functionality of photography apps.

At this point, you should have gained valuable insights and practical skills in building feature-rich photography apps using CameraX and ML Kit.

In the next chapter, we will give life to those images by learning how to capture and edit video for our Packtagram app.